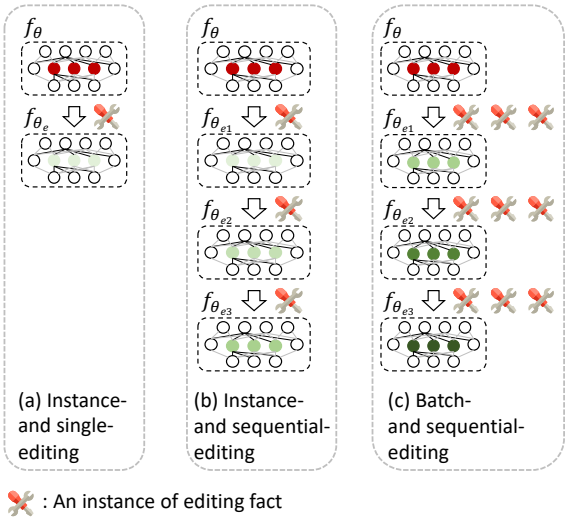

Figure 2: Illustration of the settings of (a) *single- and instance-editing*, (b) *sequential- and instance-editing*, and (c) *sequential- and batch-editing*. The darker units correspond to more edits.

is further discussed later. In practice, there are often situations where only a particular change is needed, so it's crucial to understand how effectively the model integrates and preserves that individual edit. Therefore, evaluating the robustness to a single edit is crucial in determining its ability to retain the intended changes and overall performance.

In contrast to single-editing, multiple editing operations are conducted successively in sequential-editing (Huang et al., 2023). Similarly, each editing operation in sequential-editing can also contain either only one editing instance or multiple ones in a batch. Ideally, models should retain the changes from previous edits when carrying out a new one (Yao et al., 2023), which is decisive for the continual learning of future LLMs. Therefore, whether edited models can still maintain its general abilities after sequential editing is one of the important characteristics that should be considered. For this analysis, how the performance of edited models on a variety of tasks changes as the number of edits increases will be explored.

Instance- vs. Batch-editing Instance-editing refers to using only one instance per editing operation to make specific and targeted adjustments to individual pieces of knowledge within LLMs, regardless of the single- or sequential-editing settings. This setting is particularly valuable in situations where certain instances present unique challenges or outliers that require specialized treatment. These fine-grained alterations to model behaviors over individual instances are expected to contribute to more adaptable and accurate LLMs.

The real world is ever-changing, so there is a huge amount of knowledge that needs to be dynamically added and updated into LLMs. Despite the effectiveness of many instance-editing methods (Dai et al., 2022; Meng et al., 2022; Ma et al., 2023), ultimately at most a few dozens of pieces of knowledge can be updated (Mitchell et al., 2022b), due to their relatively low but still non-negligible editing cost for a single instance. Since naive sequential applications of current state-of-the-art model editing methods fail to scale up (Meng et al., 2023), one may wish to update hundreds or thousands of facts simultaneously in batch-editing. Notably, batch-editing can also be coupled with both the single- or sequential-editing settings.

Zero-shot Learning Zero-shot learning aims to solve tasks without labeled training examples, and recent studies have demonstrated the superiority of LLMs for zero-shot learning (Brown et al., 2020; Wei et al., 2022; Chowdhery et al., 2023). Following these studies, we explore the zero-shot performance of unedited and edited models on a variety of tasks. Given a task instruction and a test problem that are concatenated as the input, the model is expected to generate a target text to address this problem. The instructions and input formats of different tasks are shown in Appendix B, which are taken from or inspired by Qin et al. (2023).

4.2 Evaluation Setup

We briefly introduced the experimental setup regarding editing methods, editing datasets, selected LLMs, and representative tasks here. Readers can refer to their corresponding papers for more details.

Editing Methods Four popular editing methods were selected: (1) *KN* (Dai et al., 2022) involved identifying neurons linked to knowledge expression using gradient-based attributions and then enhancing the MLP layer by adding scaled embedding vectors to those specific neurons. (2) *MEND* (Mitchell et al., 2022a) learned a hypernetwork to produce weight updates by decomposing the fine-tuning gradients into rank-1 form. (3) *ROME* (Meng et al., 2022) involved localizing factual knowledge in a specific Transformer MLP layer and then updating it by directly writing new key-value pairs into the MLP module. (4) *MEMIT* (Meng et al., 2023) expanded the capabilities of ROME by enabling the editing of large

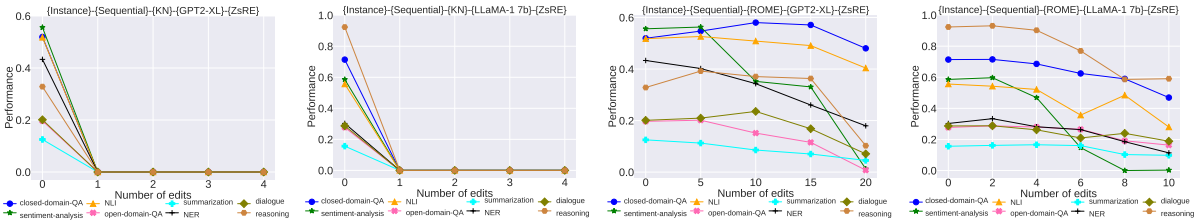

Figure 3: Performance on general tasks of edited models using KN or ROME to edit GPT-2 XL or LLaMA-1 (7B) as the number of edits increases in *instance- and sequential-editing*.

amounts of factual data through the updating of a sequence of MLP layers. It is notable that only MEND and MEMIT support batch-editing. All experiments were conducted using the EasyEdit tool (Wang et al., 2024a), ensuring standardized and reproducible evaluation. All editing instances were randomly sampled from the editing dataset. Readers can refer to Appendix C for details of these editing methods.

Editing Dataset The popular model editing dataset Zero-Shot Relation Extraction (ZsRE) (Levy et al., 2017) used in previous work (Cao et al., 2021; Meng et al., 2022; Yao et al., 2023) was adopted in our experiments. ZsRE is a QA dataset using question rephrasings generated by back-translation as the equivalence neighborhood. Each input is a question about an entity, and plausible alternative edit labels are sampled from the top-ranked predictions of a BART-base model trained on ZsRE.

Selected LLMs Experiments were conducted on three LLMs: GPT-2 XL (1.5B) (Radford et al., 2019), LLaMA-1 (7B) (Touvron et al., 2023a), and LLaMA-2 (7B) (Touvron et al., 2023b).

Downstream Tasks and Metrics To extensively explore whether model editing has side effects on the general abilities of LLMs, eight representative tasks were adopted: (1) *Reasoning* on the GSM8K (Cobbe et al., 2021), and the results were measured by solve rate. (2) *Natural language inference (NLI)* on the RTE (Dagan et al., 2005), and the results were measured by accuracy of two-way classification. (3) *Open-domain QA* on the Natural Question (Kwiatkowski et al., 2019), and the results were measured by exact match (EM) with the reference answer after minor normalization as in Chen et al. (2017) and Lee et al. (2019). (4) *Closed-domain QA* on the BoolQ (Clark et al., 2019), and the results were also measured by EM. (5) *Dialogue* on the MuTual (Cui et al., 2020), and the results were measured by selecting one best-matched response from four available candidates, denoted as $\text{Recall}_4@1$ as in Lowe et al. (2015). (6) *Summarization* on the SAMSum (Gliwa et al., 2019), and the results were measured by the average of ROUGE-1, ROUGE-2 and ROUGE-L as in Lin (2004). (7) *Named entity recognition (NER)* on the CoNLL03 (Sang and Meulder, 2003), and the results were measured by entity-level F1-score. (8) *Sentiment analysis* on the SST2 (Socher et al., 2013), and the results were measured by accuracy of two-way classification.

4.3 Results

Impact of Sequential-editing Since single-editing can be regarded as a special case of sequential-editing when the number of edits is 1, this subsection mainly discussed instance- and sequential-editing. KN and ROME that support instance-editing but not batch-editing were adopted to facilitate this exploration. MEND and MEMIT that support batch- and sequential-editing will be explored later in this subsection. Figure 3 presents the performance on general tasks of edited models using KN or ROME to edit GPT-2 XL and LLaMA-1 (7B) as the number of edits increases. Due to limited space, readers can refer to Appendix D for the results of editing LLaMA-2 (7B) which show similar trends. It can be seen that although there is only one instance per editing operation, the performance of edited models on various tasks fluctuates significantly and shows a downward trend as the number of edits increases. Strikingly, the use of KN resulted in a drastic performance degradation to nearly zero on all selected tasks with just a single edit. These findings underscore two key insights. First, the selected LLMs are not robust to weight perturbations even if less than 1% of the parameters are edited, whereby slight perturbations may significantly affect their general abilities. Second, these outcomes also shed light on the challenging nature of effectively coupling current

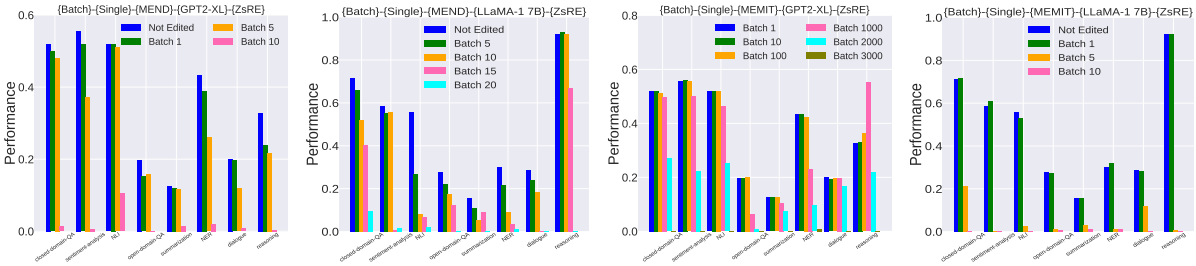

Figure 4: Performance on general tasks of edited models using MEND or MEMIT to edit GPT-2 XL or LLaMA-1 (7B) with different batch sizes in *batch- and single-editing*.

editing algorithms with LLMs. The difficulty lies in the dual objective of improving model factuality while simultaneously maintaining their general abilities. The observed trends indicate that existing editing algorithms face grand challenges in achieving this delicate balance, emphasizing the need for further research and development in the refinement of editing methodologies for LLMs.

Impact of Batch-editing This subsection delved into batch- and single-editing to explore the impact of batch size for scaling up the editing scope. Only MEND and MEMIT that supported batch-editing were adopted to facilitate this exploration. Figure 4 presents the performance on general tasks of edited models using MEND or MEMIT to edit GPT-2 XL and LLaMA-1 (7B) with different batch sizes. Readers can refer to Appendix D for the results of editing LLaMA-2 (7B). Remarkably, even with only one single editing operation, edited models exhibited a trend of performance degradation as the batch size increases in most cases. This consistent decrease in performance underlines the sensitivity of the models to increases in batch size, emphasizing the significance of carefully scaling knowledge editing for optimal updates. Therefore, we call for more research work on scalable editing to facilitate efficient editing of multiple instances.

Impact of Batch- and Sequential-editing In order to holistically take into account the interplay between batch size and sequential-editing, a joint setting of batch- and sequential-editing was explored to understand how these two factors collaboratively influence the overall performance of edited models. Figure 11 to Figure 16 in Appendix D present the performance of using MEND or MEMIT to edit GPT-2 XL, LLaMA-1 (7B) and LLaMA-2 (7B) respectively as the number of edits increases. These results also echo our observations on sequential-editing, and those on batch-editing respectively.

# edits	Manhattan dist.	% δ > 0.077	% δ > 0.171
1	9079.2	20.0%	10.0%
5	27072.2	49.2%	28.9%
10	52245.3	67.4%	46.2%
15	63247.5	72.8%	52.2%

Table 1: Statistics of the distinction between the final edited weight and the original unedited weight via weight change as the number of edits increases.

4.4 Analysis of Causes of Side Effects

We show that the side effects of editing come from changing the original model weights too much, resulting in *overfitting* to the editing facts. This phenomenon can be illustrated through statistics and visualization using ROME to edit GPT-2 XL.

Statistics We first show how the weights change in instance- and single-editing. Typically, one editing operation is to add an edit update weight ΔW to the original weight W, where ΔW is calculated aiming to insert new editing facts. Here, we define the absolute value of the relative change in weight $\delta = |\frac{\Delta W}{W}|$ to characterize the degree of change of each element in the update weight ΔW. The statistics show that only 20% of the elements in the update weight ΔW have δ greater than 0.077, while only 10% of the elements have δ greater than 0.171. These results are averaged by 100 random single edits in the ZsRE dataset. It can be seen that the update weight ΔW might be quite sparse, while most elements in ΔW are minor. Manhattan distance between the updated and original weights is also calculated as a measurement of distinction.

Furthermore, how the weights change in instance- and sequential-editing is shown in Table 1. As the number of edits increases, the proportion of elements whose δ is greater than a certain threshold increases significantly, and the weight is also more differentiated than the original weight. Therefore, the accumulation of overfitting